

MediKiosk — Hinglish PRD (Samjhaane wala version)

PS: SIH26047 — Patient Case-Taking Software (Ministry of Ayush, AIIA) **Stack:** Flutter (patient app) + Python Flask (backend) + PostgreSQL

Ye wo PRD hai jo team ke har member ko — chahe wo coder ho ya nahi — 10 minute mein product samjha de. Har section ke neeche "kaise banega" (Flask/Flutter ke hisaab se) bhi likha hai. Aakhri mein saare FAQs hain jo humne brainstorm kiye the.

1. Product ek line mein kya hai?

"Hospital ka ek smart self-service system — patient doctor ke paas jaane se PEHLE apni poori bimari ki kahani bol ke/tap karke bata deta hai, purane kaagaz scan kar deta hai, aur doctor ko ek ready-made summary mil jaati hai. Ayurvedic history (Prakriti waghera) bhi isi mein cover hoti hai."

Socho ATM/airport check-in kiosk jaisa — wahi concept, medical history ke liye.

2. Poora user journey (5 steps)

Step 1: IDENTIFY → Registration (naam, age, ABHA, language, consent) Step 2: PROFILE → Pehli baar: Prakriti assessment (10-15 min, sirf ek baar) Step 3: CONVERSE → Chat: apni takleef batao (voice ya MCQ) Step 4: SCAN → Purane documents upload → AI parse → confirm Step 5: BOOK + CONSULT → Appointment book, summary doctor ko mil gayi

3. Registration par kya-kya data lenge?

Data	Zaroori?	Kyon
Naam, DOB/age, gender	Haan	Basic identity; age auto-update hoti rahegi
Phone (OTP)	Haan	Login + alerts ke liye
Email	Nahi	Optional
Language	Haan	Isi se poori app ki bhasha + awaaz decide hogi
ABHA ID	Nahi (demo mein mock)	Enter ya scan; isse national health record se link hoga
Aadhaar	Nahi	Sirf ABHA na ho toh; masked store karenge
Emergency contact #1 (family)	Nahi (optional)	Red-flag waqt alert bhejne ke liye
Emergency contact #2 (family doctor/hospital)	Nahi (optional)	Same
Emergency pre-consent	Nahi (optional)	"Emergency mein contacts ko alert bhejin?" — haan/na
Data consent	Haan (compulsory)	Audio-guided, simple bhasha mein, kabhi bhi revoke kar sakte ho

Kaise banega (Flask/Flutter): Flutter registration form (icon + TTS audio guide) →

`POST /api/auth/register` → Flask users table mein save + OTP verify. Consent entry `consents` table mein version + timestamp ke saath.

4. Pehli interaction: Prakriti assessment (Dashavidha)

Pehli baar login ke baad app ek **guided interview** karegi — MCQ + voice dono mein. Ye CCRAS/AIIA ke validated questionnaire jaisa hoga.

Order of questions (pehli interaction)

1. **Prakriti** (Vata/Pitta/Kapha — kaunsa dominant hai)
2. **Sattva** (mental temperament — shaant/gussa/chetna)
3. **Samhanana** (body build)
4. **Sara** (tissue quality)
5. **Pramana** (height/weight)
6. **Satmya** (kya suit karta hai — diet/lifestyle habits)

Fixed vs changing — sabse important cheez

Kya?	Fixed ya badalta?	Kitni baar check?
Prakriti	Janm se fixed, kabhi nahi badalti	Ek baar. Har visit par sirf confirm ("abhi bhi Vata-Pitta hi hai na? koi badi change?")
Sattva, Samhanana	Mostly fixed	Ek baar + confirm
Sara, Pramana, Satmya	Dheere badalte hain	Saal mein ek baar refresh ya demand par
Vaya (age)	Auto	System khud update karega
Vikriti (current imbalance)	Har visit par naya	Har chat mein fresh
Agni (digestion)	Mausam/bimaari se badalta	Har visit
Koshtha (bowel)	Current state hai	Har visit
Ahara-Vihara (diet-lifestyle)	Aaj ka state	Har visit
Nidana/Samprapti (cause)	Har bimari ka alag	Har episode

Iska fayda: PS ki problem thi "repeated questioning across visits" — static data ek baar store karoge toh har baar 50 sawaal nahi poochhne padenge. Naya patient ~12 min mein profile banayega, purana patient ~30 sec ke delta-check mein.

Kaise banega: Flutter MCQ screens with voice answer → POST

/api/profile/prakriti/assessment → Flask patient_profiles table mein score (Vata/Pitta/Kapha percentages) save. Return visit par POST
/api/profile/prakriti/delta-check — 2-3 sawaal, bas.

5. Chat module — kaise shuru hogi, kaise chalegi?

Do modes:

- **General mode** (allopathic style) — complaint-first, SOCRATES probing (dard kab shuru hua? kaisa hai? kya karne se badhta hai? 0-10 mein kitna?)
- **AYUSH mode** (toggle) — extra: Vikriti, Agni, Koshtha, Ahara-Vihara, Nidana ke sawaal + stored Prakriti ka use

Flow:

1. Chat khulta hai: "Aapko kya takleef hai?" — user bol sakta hai, ya preset chips tap kar sakta hai (bukhhar, sar dard, pet dard...)
2. AI adaptive follow-up karta hai — jawab ke hisaab se agla sawaal
3. Personalization: agar profile mein diabetes hai toh "sugar kab check hui?" khud poochhega — wahi disease-list dobara nahi poochhega
4. 80% sawaal MCQ/chips honge, text almost nahi, har sawaal voice se bhi answerable

Emergency detect hua toh: Chat ke beech mein hi FR-4 flow trigger hoga (neeche detail).

Session rules: Draft auto-save; submit hote hi raw audio/transcripts **delete** — sirf structured answers database mein.

Kaise banega: Flutter chat UI (chips + mic button) → har answer POST

/api/chat/{session}/answer → Flask dialogue engine (rule-graph + LLM) agla question JSON bhejta hai → voice ke liye audio file upload → ASR transcript. chat_messages table mein sab save.

6. Emergency / Red-flag flow (poora detail)

Detection kaise hogi?

- **Rule list** (offline-safe): chest pain + saans phoolna, stroke FAST signs, khoon ki ulti, behoshi, bachche/budhe mein dehydration, etc.
- **LLM safety classifier** har answer par.
- Documents parse karte waqt bhi critical lab values flag hongii.

Sabse pehla sawaal: "Emergency mein app kaun kholega?" 🤔

Jawaab: Red-flag feature un logon ke liye hai jo **jaante hi nahi ki wo emergency mein hain**. OPD queue mein baithe log — "sirf gas hai", "umra ho gayi kamzori hai" — silent heart attack / stroke ke shuruati signs. Wo token ke liye ghante wait karenge. Hamara system unhe pakadta hai. **Jo jaanta hai ki emergency hai, wo seedha ER jaata hai — wo hamara target nahi.**

Do alag modes:

Kiosk mode (hospital premises par):

- Alert usi hospital ki **triage desk** par — screen popup + sound: "Kiosk 3 — Chest pain + saans phoolna, Male 58"
- Staff attendant kiosk par aata hai, patient ko queue se nikaal ke priority deta hai
- Consent issue nahi — patient usi hospital mein care ke liye aaya hai

App mode (ghar par):

- **Non-skippable emergency screen**, 3 bade buttons:
 1. 📞 **108 call karo** (kisi contact ki zaroorat nahi)
 2. 👤 **Emergency contacts ko alert** (SMS/WhatsApp — minimal data + location link)
 3. 📍 **Nearest ER ka route** (GPS navigation)
- Saath mein **first-aid guidance** (CPR, recovery position, stroke FAST) — bina kisi data sharing ke
- **Auto-action fallback:** 10 second koi tap nahi → contacts hain toh auto-alert, nahi toh auto-dial 108

"Triage staff alert kisko jayega agar naya user hai, aur wo bhi bina permission?"

3-layer jawaab:

1. **Kiosk user** → usi hospital ki desk par (koi confusion nahi)
2. **App user** → registration ke time optional pre-consent liya tha ("emergency mein contacts ko alert kar dein?") + event ke time one-tap confirm
3. **User response na de paye** → **DPDP Act 2023 Section 7(c)**: medical emergency (life threat) mein consent ki zaroorat nahi hoti — legal exception. Alert bhej do.
4. **Alert payload minimal hoga**: sirf flag + naam + age + kiosk ID/GPS — poori history kabhi nahi bhejenge.

"Koi contact add hi nahi kiya toh?"

- **108 ka button hamesha hai** — contact list ki zaroorat hi nahi
- Screen par 108 + nearest ER route primary ban jaate hain
- Auto-action: 10 sec mein kuch nahi → auto-dial 108
- **Matlab koi user kabhi akela nahi chhootta.**

Red flag ≠ Urgent booking (dono alag hain)

- **Urgent booking** = patient ko *lagta hai* jaldi hai (fever, fracture) → normal booking, bas priority slot
- **Red flag** = *system* ko lagta hai khatra hai (patient ko pata bhi nahi) → booking bypass, seedha FR-4 flow

Kaise banega: Chat engine har answer par rules check karega → trigger par Flutter emergency screen (kiosk mode mein `POST /api/alerts/triage`) → Flask `alerts` + `audit_logs` table mein entry → app mode mein contacts alert via SMS gateway (demo mein mock).

7. History page — documents ka workflow

User kya karega:

1. Camera/gallery se document capture (prescription, lab report, discharge summary — PDF bhi chalega)

2. Upload → backend par pipeline chalti hai: `Preprocess (image saaf) → OCR (Hindi+English, handwritten bhi) → LLM extraction → Structured JSON → User ko review screen`
3. User review kare: har item icon-card mein — "Dawai: Metformin 500mg — roz 2 baar ✓/✗" → **Confirm / Fix / Skip**
4. Skip kiya toh status `unverified` — doctor ko "Unverified" badge ke saath dikhega
5. **Store: original file + parsed JSON** — dono saath, ek record
6. Timeline: saare docs + past visits reverse-chronological order mein

Kya-kya parse karna hai?

Document	Kya nikalenge
Prescription	doctor ka naam/specialty, date, dawaiyan (naam, dose, frequency, duration, instructions), diagnosis hints
Lab report	lab naam, date, tests (naam, value, unit, ref range, normal/high/low/critical flag)
Discharge summary	hospital, admit/discharge dates, diagnoses, procedures, medicines, advice, follow-up
Generic	type, patient name match, date, key entities, confidence

Doctor-facing extras: abnormal lab values highlight, drug-drug interaction flags, duplicate medicine warning.

Kaise banega: Flutter camera/file picker → `POST /api/documents` → Flask Celery job (PaddleOCR/EasyOCR → LLM JSON-schema extraction) → `GET /api/documents/{id}/parsed` Flutter review screen → `POST /api/documents/{id}/confirm` → `documents` table (`original_path` + `parsed_json`) save.

8. Appointments — urgent vs regular

 **Note:** Appointment booking PS mein actually hai hi nahi (PS ka scope sirf pre-consult history capture hai). Ye hamari **value-add** hai jo journey complete karti hai aur summary doctor tak pahunchati hai. Demo mein slim rakhna, zyada mat phoolana.

Flow:

1. **Discover:** nearest partner hospitals/doctors (GPS) — demo mein 5-10 seeded entries
2. **Book:** hospital → doctor → slot → **Regular ya Urgent**
 - Regular = normal slots
 - Urgent = same-day priority slot + reason chip + staff flag
3. **Attach medical context:** current AI summary + selected parsed records
4. **Consent picker:** "Is doctor ko kya dikhega?" → Summary only / Summary + documents / Full record
5. Confirm → token + queue status app mein
6. Appointment record mein summary snapshot attach; **mock FHIR Bundle** bhi banta hai jo HIS/ABHA ko "push" hota hai (demo mein stub)

Doctor ka access:

- Access **appointment start par khulta hai, end par auto-revoke**
- Patient kabhi bhi pehle revoke kar sakta hai (Settings → Consents)
- Doctor ne kab kya khola — sab audit log mein

Patient ki appointment history:

- Upcoming/past lists, cancel/reschedule, har appointment mein "kya data share kiya tha" dikhega

Kaise banega: Flask `appointments`, `hospitals`, `doctors` tables + REST APIs; Flutter booking wizard; doctor portal Flask-served pages; consent window logic backend par enforce hogi (UI par nahi — security backend par).

9. Summary generation — doctor tak kaise pahunchti hai?

Inputs:

- Current visit ke chat answers (structured)
- AYUSH dynamic params (Vikriti, Agni, Koshtha, Nidana)
- Static profile (Prakriti, allergies, past history)

- Verified parsed documents

Output structure:

1. Chief complaint 2. History of Present Illness 3. Past medical/surgical history
4. Drug & allergy 5. Family history 6. Personal habits 7. Review of Systems 8. Prior investigations (abnormal values highlighted) 9. AYUSH block (Prakriti, Vikriti, Agni, Koshtha, Ahara-Vihara, Nidana) 10. Red flags (agar koi) 11. Disclaimer: "AI-generated draft – diagnosis nahi hai"

Delivery:

- **Patient ko:** local language mein audio confirm — "Aapki jankari record ho gayi"
- **Doctor ko:** English/Hindi toggle ke saath summary screen, **Accept / Amend / Reject** — doctor ka final version wapas record mein save hoga (versioning ke saath)

Sharing (appointment ke dauran):

1. Summary appointment record se attach (doctor portal mein dikhegi)
2. Mock FHIR push (HIS/ABHA) — demo mein stub, production mein real adapter
3. Doctor sirf appointment window ke andar, consent scope ke hisaab se, audit-logged access

Kaise banega: `POST /api/summaries/generate` → Flask summarizer (LLM ya Jinja template fallback) → `summaries` table (versioned) → doctor portal `GET /api/doctor/appointments/{id}/summary` → edits `POST /api/summaries/{id}/doctor-edit`.

10. Consent — grant aur revoke ka poora lifecycle

Kab	Kya consent	Kaise
Registration	Data capture consent	Audio-guided TTS + tap Yes/No, plain language
Registration (optional)	Emergency contact alert pre-consent	Toggle + disclaimer ("contacts add karte waqt unki sehmati lein")
Appointment booking	Granular share consent	Picker: Summary only / + docs / Full record, per-doctor, time-bound
Appointment end	Auto-revoke	System khud revoke karega (ya slot ke 24h baad, configurable)
Kabhi bhi	Manual revoke	Settings → Consents → Revoke (turant effect)
Emergency	DPDP §7(c) exception	Consent ke bina bhi allowed, audit log mein documented

- Har consent `consents` table mein ledger entry: scope, purpose, granted_at, expires_at, revoked_at, version
- **Audit log:** kaunse doctor ne kab kya dekha — append-only, admin exportable
- **Security:** TLS, JWT, AES-256 at rest, OTP, session data wipe (raw audio/transcripts submission ke baad delete)

11. Doctor side (Flask-served web portal)

1. **Summary screen** — 1 minute se kam mein padh sake
2. **Accept/Amend/Reject** + version history
3. **Source documents panel** — original file + parsed data side-by-side, unverified badges, abnormal highlights, drug flags
4. **Patient timeline** — past visits + documents chronological
5. **AYUSH exam form** — Ashtavidha ke physical checks jo AI nahi kar sakta: **Nadi (pulse), Jihva (tongue), Mutra, Mala, Sparsha (touch), Drik (eyes), Akriti (build)** — doctor structured form mein bharega, isse Dashavidha complete hogi

12. Demo plan (90 second story)

1. Hindi mein register (audio-guided) → 2. Prakriti assessment (6-8 MCQ) → 3. Chat: "pet mein dard" → adaptive follow-ups → 4. Ek seeded document upload → parse preview → confirm → 5. Summary generate (Hindi/English) → 6. Appointment book + consent scope choose → 7. Doctor portal: summary + docs + accept/amend → 8. **Red-flag demo:** chest pain pattern → emergency screen + triage alert fire

Sab offline chalega — seeded data, rule-based fallbacks, mock ABHA/FHIR. Judges ke liye KPI bolna: "intake time X min, doctor read-time 5 min → 30 sec".

13. FAQs (jo humne brainstorm kiye, sab jawaab ke saath)

Q1. Ayurveda details user se kaise nikalenge? A: Do hisse. (1) Questionnaire-based Dashavidha items (Prakriti, Sattva, Samhanana, Sara, Pramana, Satmya) — AI voice+MCQ interview se, CCRAS/AIIA validated scales par. (2) Physical items (Nadi, Jihva, Mutra, Mala, Sparsha, Drik, Akriti) — AI nahi kar sakta, doctor ko structured form denge jo uske consultation screen par hai. Dono milkar complete Dashavidha.

Q2. Prakriti fixed hai ya baar-baar check karna padega? A: Prakriti janm se fixed hai — kabhi nahi badalti. Ek baar assess karke profile mein save; har visit par sirf 10-30 sec ka delta-check ("koi badi change hui?"). Lekin Vikriti, Agni, Koshtha, Ahara-Vihara, Nidana har visit par fresh chahiye. Isliye do-layer data model: static profile + dynamic visit record.

Q3. Emergency mein aadmi app khol ke chat karega? A: Wo jo jaanta hai ki emergency hai — nahi karega, wo seedha ER jayega. Hamara target wo log hain jo **nahi jaante** ki wo emergency mein hain (silent heart attack, stroke ke shuruati signs) — queue mein baithe log. Unhe system pakadta hai.

Q4. Naye user ka red-flag alert kisko jayega, bina permission ke? A: Kiosk mode mein usi hospital ki triage desk par (patient wahan care ke liye aaya hai). App mode mein: registration ke time optional pre-consent + event par one-tap confirm. Agar user response na de paye — DPDP Act 2023 Section 7(c) emergency exception se bina consent ke bhi allowed hai. Har alert audit-logged.

Q5. User ne koi emergency contact add nahi kiya toh? A: 108 button hamesha hai — contact list ki zaroorat nahi. No-contact case mein 108 call + nearest ER route primary options ban jaate hain. 10 second no-tap par auto-dial 108. Koi bhi user akela nahi chhootta.

Q6. Urgent booking aur red flag mein kya fark hai? A: Urgent booking = patient ko lagta hai jaldi hai → normal flow, priority slot. Red flag = system ko lagta hai khatra hai (patient ko pata bhi nahi) → booking bypass, seedha triage/emergency flow. Dono alag layers, kabhi mix nahi.

Q7. Document upload ke baad asli mein kya hota hai? A: Upload → image preprocessing → OCR (Hindi+English, handwritten bhi) → LLM extraction (diagnoses, medicines with doses, lab values with ranges, procedures) → user review screen (confirm/fix/skip) → original file + parsed JSON store → timeline mein chronological add → abnormal values aur drug interactions flag (doctor ke liye).

Q8. Doctor ko summary kahan dikhegi? A: Flask-served doctor web portal par — appointment ke andar. Doctor Accept/Amend/Reject kar sakta hai, source documents dekh sakta hai, patient timeline dekh sakta hai. Access sirf appointment window ke andar, consent scope ke hisaab se, audit-logged.

Q9. Consent kab grant hota hai, kab revoke? A: Registration par data capture consent (audio-guided). Appointment booking par granular share consent (summary only / +docs / full record) specific doctor ke liye. Appointment end par **auto-revoke**. Patient kabhi bhi Settings se manual revoke kar sakta hai. Emergency mein DPDP §7(c) exception.

Q10. Appointment booking PS mein hai hi nahi — phir kyun bana rahe hain? A: Sahi pakda. Ye hamari value-add hai jo journey complete karti hai (summary doctor tak pahunchane ka natural carrier). Isliye demo mein slim rakhna aur pitch mein "value-add beyond PS" bolna — PS ka core (history capture + digitization + summary + consent) pehle dikhna chahiye.

Q11. AI galat data parse kar de toh? A: Human-in-the-loop — user confirm karta hai; skip par "unverified" badge doctor ko dikhta hai; doctor bhi edit kar sakta hai. AI kabhi final authority nahi — doctor accept/amend/reject karta hai. Disclaimer har summary par.

Q12. Internet na ho toh? A: Poora demo offline-safe hai — rule-based dialogue fallback, template summarizer, seeded documents, mock ABHA/FHIR. Production mein offline queue + sync.

Q13. Judges ke paas ABHA/HIS APIs nahi honge? A: Mock adapters — ABHA QR simulation, FHIR Bundle generation (valid JSON dikhana), HIS push stub. Architecture mein real integration ka hook documented.

Q14. Data kahan store hota hai, safe hai? A: PostgreSQL (users, profiles, chats, documents, consents, audit). Original files S3-compatible bucket mein. TLS in transit, AES-256 at rest, JWT auth, OTP, Aadhaar/ABHA masked. Session khatam hote hi raw audio/transcripts wipe.

Q15. Budhe/anpadh log kaise use karenge? A: Icon-driven UI, 48px+ touch targets, har sawaal TTS se bol ke sunaya jayega, voice se jawab, auto-advance, audio-only completion path, aur kiosk par "attendant bulao" button.

14. Tech-stack quick map

Kaam	Kaise
Patient app	Flutter (mobile + kiosk responsive)
Backend	Flask REST + JWT
DB	PostgreSQL + SQLAlchemy
Async jobs (OCR/summary)	Celery + Redis
Voice → text	Bhashini/AI4Bharat (prod) / Vosk fallback (demo)
Text → awaaz	flutter_tts
OCR	PaddleOCR/EasyOCR + Tesseract fallback
Extraction/dialogue/summary	LLM (JSON-schema constrained) + rule/template fallback
Interop	FHIR mock adapter (prod: real ABDM sandbox)
File storage	Local (dev) → S3 (prod)

15. Kaunsa order mein banao (demo priority)

1. Registration + consent screens

2. Prakriti assessment flow
 3. Chat (rule-based graph + chips + voice)
 4. Summary generation (template/LLM)
 5. Doctor portal (summary + accept/amend)
 6. Document upload + parse + confirm
 7. Appointments + consent windows + auto-revoke
 8. Red-flag/emergency flow (last mein bhi chalega, par demo ka hero hai)
-

Bas itna hi — ab team ko bhej do, sab samajh jayenge. 🚀